

## **Limbajul de definire a datelor (LDD) - II :** **Definirea vizualizărilor, secvențelor**

### **I. Definirea vizualizărilor (view)**

- **Vizualizările sunt tabele virtuale** construite pe baza unor tabele sau a altor vizualizări, denumite tabele de bază.
- **Vizualizările nu conțin date, dar reflectă datele din tabelele de bază.**
- Vizualizările sunt definite de o cerere SQL, motiv pentru care mai sunt denumite **cereri stocate**.

➤ **Avantajele** utilizării vizualizărilor:

- restricționarea accesului la date;
- simplificarea unor cereri complexe;
- asigurarea independenței datelor de programele de aplicații;
- prezentarea de diferite imagini asupra datelor.

➤ **Crearea vizualizărilor** se realizează prin comanda *CREATE VIEW*, a cărei sintaxă simplificată este:

```
CREATE [OR REPLACE] [FORCE | NOFORCE] VIEW  
           nume_vizualizare [(alias, alias, ..)]  
AS subcerere  
[WITH CHECK OPTION [CONSTRAINT nume_constrangere]]  
[WITH READ ONLY [CONSTRAINT nume_constrangere]];
```

- *OR REPLACE* se utilizează pentru a schimba definiția unei vizualizări fără a mai reacorda eventualele privilegii.
- Opțiunea *FORCE* permite crearea vizualizării înainte de definirea tabelelor, ignorând erorile la crearea vizualizării.
- Subcererea poate fi oricât de complexă dar **nu poate conține clauza *ORDER BY***. Dacă se dorește ordonare se utilizează *ORDER BY* la interogarea vizualizării.
- *WITH CHECK OPTION* permite inserarea și modificarea prin intermediul vizualizării numai a liniilor ce sunt accesibile vizualizării. Dacă lipsește numele constrângerii atunci sistemul asociază un nume implicit de tip *SYS\_Cn* acestei constrângeri (*n* este un număr astfel încât numele constrângerii să fie unic).
- *WITH READ ONLY* asigură că prin intermediul vizualizării **nu se pot executa operații LMD**.

➤ **Modificarea vizualizărilor** se realizează prin recrearea acestora cu ajutorul opțiunii *OR REPLACE*. Totuși, începând cu *Oracle9i*, este posibilă utilizarea comenzii *ALTER VIEW* pentru adăugare de constrângeri vizualizării.

➤ **Suprimarea vizualizărilor** se face cu comanda *DROP VIEW* :  
**DROP VIEW *nume\_vizualizare*;**

- Informații despre vizualizări se pot găsi în [dicționarul datelor](#) interogând vizualizările: [USER\\_VIEWS](#), [ALL\\_VIEWS](#). Pentru aflarea informațiilor despre coloanele actualizabile, este utilă vizualizarea [USER\\_UPDATABLE\\_COLUMNS](#).
- Subcererile însoțite de un alias care apar în comenzile *SELECT*, *INSERT*, *UPDATE*, *DELETE* se numesc **vizualizări inline**. Spre deosebire de vizualizările propriu zise, acestea nu sunt considerate obiecte ale schemei ci sunt entități temporare (valabile doar pe perioada execuției instrucțiunii LMD respective).
- **Operații LMD asupra vizualizărilor**
  - Vizualizările se pot împărți în **simple** și **complexe**. Această clasificare este importantă pentru că **asupra vizualizărilor simple se pot realiza operații LMD**, dar în cazul celor complexe acest lucru nu este posibil întotdeauna (decât prin definirea de *triggeri* de tip *INSTEAD OF*).
    - o **Vizualizările simple** sunt definite pe baza unui singur tabel și **nu conțin funcții sau grupări de date**.
    - o **Vizualizările compuse** sunt definite pe baza mai multor tabele sau conțin funcții sau grupări de date.
  - **Nu se pot realiza operații LMD** în vizualizări ce conțin:
    - funcții grup,
    - clauzele *GROUP BY*, *HAVING*, *START WITH*, *CONNECT BY*,
    - cuvântul cheie *DISTINCT*,
    - pseudocoloana *ROWNUM*,
    - operatori pe mulțimi.
  - **Nu se pot actualiza:**
    - coloane ale căror valori rezultă prin calcul sau definite cu ajutorul funcției *DECODE*,
    - coloane care nu respectă constrângerile din tabelele de bază.
  - **Pentru vizualizările bazate pe mai multe tabele**, orice operație *INSERT*, *UPDATE* sau *DELETE* poate **modifica datele doar din unul din tabelele de bază**. Acest tabel este cel protejat prin cheie (**key preserved**). În cadrul unei astfel de vizualizări, un tabel de bază se numește *key-preserved* dacă are proprietatea că fiecare valoare a cheii sale primare sau a unei coloane având constrângerea de unicitate, este unică și în vizualizare.  
Prima condiție ca o vizualizare a cărei cerere conține un *join* să fie modificabilă este ca instrucțiunea LMD să afecteze un singur tabel din operația de *join*.
- Reactualizarea tabelelor implică reactualizarea corespunzătoare a vizualizărilor!!!

### Exerciții [I]

1. Pe baza tabelului *EMP\_PNU*, să se creeze o vizualizare *VIZ\_EMP30\_PNU*, care conține codul, numele, email-ul și salariul angajaților din departamentul 30. Să se analizeze structura și conținutul vizualizării. Ce se observă referitor la constrângeri? Ce se obține de fapt la interogarea conținutului vizualizării? Inerați o linie prin intermediul acestei vizualizări; comentați.
  2. Modificați *VIZ\_EMP30\_PNU* astfel încât să fie posibilă inserarea/modificarea conținutului tabelului de bază prin intermediul ei. Inerați și actualizați o linie (cu valoarea 300 pentru codul angajatului) prin intermediul acestei vizualizări.
- Obs:** Trebuie introduse neapărat în vizualizare coloanele care au constrângerea *NOT NULL* în tabelul de bază (altfel, chiar dacă tipul vizualizării permite operații LMD, acestea nu vor fi posibile din cauza nerespectării constrângerilor *NOT NULL*).

Unde a fost introdusă linia? Mai apare ea la interogarea vizualizării?

Ce efect are următoarea operație de actualizare?

```
UPDATE viz_emp30_pnu SET hire_date=hire_date-15
WHERE employee_id=300;
```

Comentați efectul următoarelor instrucțiuni, analizând și efectul asupra tabelului de bază:

```
UPDATE emp_pnu SET department_id=30
WHERE employee_id=300;

UPDATE viz_emp30_pnu SET hire_date=hire_date-15
WHERE employee_id=300;
```

Ștergeți angajatul având codul 300 prin intermediul vizualizării. Analizați efectul asupra tabelului de bază.

3. Să se creeze o vizualizare, *VIZ\_EMPSAL50\_PNU*, care conține coloanele *cod\_angajat*, *nume*, *email*, *functie*, *data\_angajare* și *sal\_anual* corespunzătoare angajaților din departamentul 50. Analizați structura și conținutul vizualizării.
4. a) Inserați o linie prin intermediul vizualizării precedente. Comentați.  
b) Care sunt coloanele actualizabile ale acestei vizualizări? Verificați răspunsul în dicționarul datelor (*USER\_UPDATABLE\_COLUMNS*).
- c) Inserați o linie specificând valori doar pentru coloanele actualizabile.  
d) Analizați conținutul vizualizării *viz\_empsal50\_pnu* și al tabelului *emp\_pnu*.
5. Să se creeze vizualizarea *VIZ\_DEPT\_SUM\_PNU*, care conține codul departamentului și pentru fiecare departament salariul minim, maxim și media salariilor. Ce fel de vizualizare se obține (complexă sau simplă)? Se poate actualiza vreo coloană prin intermediul acestei vizualizări?
6. a) Definiți o vizualizare, *VIZ\_EMP\_S\_PNU*, care să conțină detalii despre angajații corespunzători departamentelor care încep cu litera S. Se pot insera/actualiza linii prin intermediul acestei vizualizări? În care dintre tabele? Ce se întâmplă la ștergerea prin intermediul vizualizării?  
b) Recreați vizualizarea astfel încât să nu se permită nici o operație asupra tabelelor de bază prin intermediul ei. Încercați să introduceți sau să actualizați înregistrări prin intermediul acestei vizualizări.
7. Să se creeze o vizualizare *VIZ\_SAL\_PNU*, ce conține numele angajaților, numele departamentelor, salariile și locațiile (orașele) pentru toți angajații. Etichetați sugestiv coloanele. Considerați ca tabele de bază tabelele originale din schema HR. Care sunt coloanele actualizabile?
8. Să se implementeze în două moduri constrângerea ca numele angajaților nu pot începe cu șirul de caractere „Wx”.

#### Metoda 1:

```
ALTER TABLE emp_pnu
ADD CONSTRAINT ck_name_emp_pnu
CHECK (UPPER(last_name) NOT LIKE 'WX%');
```

#### Metoda 2:

```
CREATE OR REPLACE VIEW viz_emp_wx_pnu
AS SELECT *
FROM emp_pnu
WHERE UPPER(last_name) NOT LIKE 'WX%'
WITH CHECK OPTION CONSTRAINT ck_name_emp_pnu2;

UPDATE viz_emp_wx_pnu
SET nume = 'Wxyz'
WHERE employee_id = 150;
```

## II. Definirea secvențelor

- Secvența este un obiect al bazei de date ce permite generarea de întregi unici pentru a fi folosiți ca valori pentru cheia primară sau coloane numerice unice. Secvențele sunt independente de tabele, așa că aceeași secvență poate fi folosită pentru mai multe tabele.
- **Crearea secvențelor** se realizează prin comanda *CREATE SEQUENCE*, a cărei sintaxă este:

```
CREATE SEQUENCE nume_secv  
[INCREMENT BY n] [START WITH n] [{MAXVALUE n | NOMAXVALUE}]  
[{MINVALUE n | NOMINVALUE}] [{CYCLE | NOCYCLE}]  
[{CACHE n | NOCACHE}]
```

La definirea unei secvențe se pot specifica:

- numele secvenței
  - diferența dintre 2 numere generate succesiv, implicit fiind 1 (*INCREMENT BY*);
  - numărul initial, implicit fiind 1 (*START WITH*);
  - valoarea maximă, implicit fiind  $10^{27}$  pentru o secvență ascendentă și  $-1$  pentru una descendentă;
  - valoarea minimă, implicit fiind 1 pentru o secvență ascendentă și  $-10^{27}$  pentru o secvență descendentă;
  - dacă secvența ciclează după ce atinge limita; (*CYCLE*)
  - câte numere să încarce în *cache server*, implicit fiind încărcate 20 de numere (*CACHE*).
- Informații despre secvențe găsim în dicționarul datelor. Pentru secvențele utilizatorului curent, interogăm *USER\_SEQUENCES*. Alte vizualizări utile sunt *ALL\_SEQUENCES* și *DBA\_SEQUENCES*.
  - **Pseudocoloanele *NEXTVAL* și *CURRVAL*** permit lucrul efectiv cu secvențele.
    - *Nume\_secv.NEXTVAL* - returnează următoarea valoare a secvenței, o valoare unică la fiecare referire. Trebuie aplicată cel puțin o dată înainte de a folosi *CURRVAL*;
    - *Nume\_secv.CURRVAL* – obține valoarea curentă a secvenței.

**Obs:** Pseudocoloanele se pot utiliza în:

- lista *SELECT* a comenzilor ce nu fac parte din subcereri;
- lista *SELECT* a unei cereri ce apare într un *INSERT*;
- clauza *VALUES* a comenzii *INSERT*;
- clauza *SET* a comenzii *UPDATE*.

**Obs:** Pseudocoloanele nu se pot utiliza:

- în lista *SELECT* a unei vizualizări;
- într-o comandă *SELECT* ce conține *DISTINCT*, *GROUP BY*, *HAVING* sau *ORDER BY*;
- într-o subcerere în comenzile *SELECT*, *UPDATE*, *DELETE*
- în clauza *DEFAULT* a comenzilor *CREATE TABLE* sau *ALTER TABLE*.

- **Ștergerea secvențelor** se face cu ajutorul comenzii *DROP SEQUENCE*.  
**DROP SEQUENCE** nume\_secventa;

**Exerciții [II]**

9. Creați o secvență pentru generarea codurilor de departamente, *SEQ\_DEPT\_PNU*. Secvența va începe de la 400, va crește cu 10 de fiecare dată și va avea valoarea maximă 10000, nu va cicla și nu va încărca nici un număr înainte de cerere.
10. Creați o secvență pentru generarea codurilor de angajați, *SEQ\_EMP\_PNU*. Să se modifice toate liniile din *EMP\_PNU* (dacă nu mai există, îl recreați), regenerând codul angajaților astfel încât să utilizeze secvența *SEQ\_EMP\_PNU* și să avem continuitate în codurile angajaților.
11. Ștergeți secvența *SEQ\_DEPT\_PNU*.